

# Catch the Falling Objects Lab - Chat History

User prompts and assistant replies; terminal/tool output and interim progress messages omitted.

Generated 2026-09-30

## User

bro these are the requirements I have to do a lab of my week 4 and what I need to do with this assist me to complete this lab and

## Assistant

The lab workflow is: clone/fork the assigned repository, run the original Python game, record a 10-second before video, use an LLM iteratively to fix the four tasks one by one, record a 10-second after video, and push the completed work to the personal fork. The four tasks are collision timing/list mutation, basket boundaries and smooth control, controlled spawning, and a temporary speed boost.

## User

broo I did fork the repo what to do next

## Assistant

After forking, clone your fork locally, enter the catch-falling directory, install the requirements, and run the original game. Do not edit the code before recording the before video. Use Shift + Command + 5 on macOS to record about 10 seconds of the original game.

## User

broo I got these many files

## Assistant

That is the expected project structure. The repository contains game/, main.py, and requirements.txt. Next, inspect the game folder and the original collision/game-engine files before making changes.

## User

its given like this

## Assistant

The game failed because Pygame was not installed. Use a virtual environment instead of changing the system Python installation. Create and activate a venv, install requirements, and run main.py.

## User

game was started soo I need to make video right

## Assistant

Yes. Record the 10-second BEFORE video now, before making any code changes. Try to show the original early-catch behavior. Save it as before.mov if that is the format macOS provides.

## User

yeah I done with the before video soo what next

## Assistant

Start Task 1. The collision function currently checks only horizontal position, and the catch loop removes objects from the same list it is iterating over. Inspect falling\_object.py and basket.py so the collision fix can account for the actual object radius and basket rectangle.

## User

soo exactly what I need to run

## Assistant

Run the commands to inspect falling\_object.py and basket.py. Also make a Git checkpoint before changes so there is a clean starting point.

**User**

[Shared the falling object and basket code]

**Assistant**

The object has x, y and radius 14. The basket is 90 pixels wide and 24 pixels high, and get\_rect() represents the basket rectangle. This is enough to implement Task 1 safely.

**User**

yeah broo I think task 1 is done while in game when we place the table below ball then only it disappear and count as 1

**Assistant**

That confirms the vertical collision behavior is fixed. The catch loop was also changed to iterate over a copy of the objects list, so adjacent catchable objects will not be skipped.

**User**

check broo

**Assistant**

The current game\_engine.py still had the original fixed spawning and basic movement sections, but the Task 1 loop correctly uses self.objects[:]. Next is Task 2: keep the basket fully inside the screen and maintain responsive held-key movement.

**User**

yeah broo done with this

**Assistant**

Task 2 is complete and committed. The basket boundary now accounts for its own width, so its center is clamped between basket.width/2 and WIDTH - basket.width/2.

**User**

[Shared the renderer code]

**Assistant**

The renderer confirms WIDTH = 700 and HEIGHT = 500. Since the falling object radius is 14, spawning should keep the object's center between 14 and WIDTH - 14. Task 3 can now add varied timing, varied positions, minimum spawn distance, and a maximum object count.

**User**

broo below this I need add or what

**Assistant**

For the Task 3 spawn block, replace the old fixed-interval block rather than adding another block below it. Use a random interval between MIN\_SPAWN\_INTERVAL\_FRAMES and MAX\_SPAWN\_INTERVAL\_FRAMES, while checking len(self.objects) < MAX\_OBJECTS.

**User**

but before you said add init

**Assistant**

Yes. The \_\_init\_\_ change is separate. Add self.last\_spawn\_x = None in \_\_init\_\_, replace \_spawn\_object() with logic that avoids repeatedly spawning near the previous x position, and replace the old fixed spawn block with the randomized, capped version.

**User**

broo see this is actually I have know tell me what to do

### Assistant

The constants had been changed, but `last_spawn_x`, the new `_spawn_object()`, and the new `update()` spawn block were still missing. Complete those three parts, compile the file, run the game, and test positions, timing, and the five-object maximum.

### User

broo give me the whole code soo directly I. will replaced that

### Assistant

A complete Task 1 + Task 2 + Task 3 `game_engine.py` was provided so it could be replaced safely in one step. The version kept the collision-copy loop, the basket boundary clamp, and added controlled spawning with randomized intervals and a five-object cap.

### User

see this is exactly hat I have

### Assistant

The current `game_engine.py` contains Tasks 1-3. For Task 4, do not discard those changes. Add the temporary speed boost on top of the existing version, using `basket.py` for the boost state/timer and `game_engine.py` for activation and display.

### User

[Shared `main.py`]

### Assistant

`main.py` already forwards `KEYDOWN` events to `engine.handle_keydown(event.key)`, so Spacebar can be used for boost activation. No `main.py` change is necessary for the feature.

### User

yeh broo done with this

### Assistant

Task 4 is committed. The boost activates with Space, doubles the basket speed temporarily, displays `BOOST ACTIVE!`, and automatically returns to normal speed after the timer expires.

### User

broo can you check once everything is perfect or not

### Assistant

The final gameplay was checked. The four task behaviors were visible, but the after video was longer than the requested 10 seconds and the boost label could remain visible after Game Over. A small draw-condition fix was recommended, followed by recording a clean approximately 10-second after video.

### User

yeah I done soo what next

### Assistant

Create a Lab-4 folder containing `before.mov`, `after.mov`, `Chat_History.pdf`, and `Updated_Code`. Copy only the actual project files into `Updated_Code`, excluding the virtual environment and macOS `.DS_Store` files. Then commit the Lab-4 folder and push the main branch to the personal fork, not the main repository.

### User

see once

### Assistant

The Lab-4 folder already contained `before.mov`, `after.mov`, and `Updated_Code`, and `Updated_Code` contained `game/`, `main.py`, and `requirements.txt`. The remaining deliverable was the chat history PDF.

### User

broo actually It contains 67 pages soo I don't need like Thai I shown you my frnd history soo it will be like that soo create you only pdf and give to m e

### **Assistant**

The chat history can be kept in the same simple style as the reference: a title, generated date, then User and Assistant sections in sequence, while omitting terminal/tool output and other interim material. The content should be this Catch the Falling Objects lab conversation, not the friend's Defender Lab content.